CS4433 INDIVIDUAL ASSIGNMENT

Student number: 202321241

5. Software Design Document

5.1 Purpose and Design Inputs

This section presents the design of the Moderated Learning Management System, release v1.0. It turns the requirements of Section 4 into an architecture, a class design, behavioural models and a database design. The requirement IDs (FR, NFR), business rules (BR), assumptions (AS) and use cases (UC) used here are those defined in Sections 4.2 and 4.3, so each design element can be traced back to the SRS (Section 5.6). Diagrams follow UML 2.5 and are drawn from PlantUML text sources that will be kept in the Git repository with the code.

The design is shaped mainly by the requirements below. Each row names the design response, and the section where it is described.

Requirement	Design response	Section
FR02 (server-side authorisation)	All rules and permission checks run in the server. The client has no business logic and no database access.	5.2.1, 5.3.1
NFR06 (works offline)	One server on the school LAN with dependency-light libraries (cpp-httplib, nlohmann::json, SQLite, libsodium). No call leaves the school network.	5.2.1
NFR04 (Argon2id password hashes)	Hashing through libsodium. The number of simultaneous hash operations is capped to protect memory use and response time.	5.2.6
NFR08, NFR09 (transactions, audit)	Every data-changing operation runs inside one transaction. The audit row is written in the same transaction as the mark change. Triggers make the audit table append-only.	5.2.5, 5.5.5
NFR01, NFR02, NFR11 (speed, memory)	SQLite in WAL mode, pooled read connections and one serialised writer, indexes on the common queries, files kept on disk instead of in the database, sessions held in memory, console client.	5.2.5, 5.5
NFR12 (70% coverage)	Services depend on interfaces (repositories, clock, penalty policy), so unit tests can use in-memory fakes and a fixed clock.	5.3.3
BR3, FR18 (late penalty)	One Strategy interface, IPenaltyPolicy, used by both marking and moderation. No API accepts a penalty value from a user.	5.3.2
BR5, FR19 (life cycle)	One class, SubmissionLifecycle, decides which status changes are legal. The database also rejects impossible status and mark combinations.	5.4.1, 5.5.5

5.2 Architecture

5.2.1 Architectural Style

The system uses a **layered client–server architecture**. One server process owns the SQLite database and the submission files. Console clients on the school PCs talk to it over the school LAN with HTTP and JSON. Inside the server the code is arranged in three layers (API, business logic, data access). Each layer may call only the layer directly below it. The business logic layer depends on repository interfaces and not on SQLite classes, so the source-code dependencies point towards the business

rules (the Dependency Inversion Principle). A single composition root in `main()` creates the concrete objects and passes them to the services.

Layer	Responsibility	Must not
Client (console)	Show role-specific menus, collect input, send requests, display results and error messages.	Hold business rules, open the database, use its own clock.
API layer (server)	Parse HTTP, authenticate the session, build the User object, call one service method, map exceptions to HTTP status codes.	Contain business rules or SQL.
Business logic layer	Authorisation (role and scope, BR9), validation, life-cycle and penalty rules, transaction boundaries, audit.	Know about HTTP or SQL.
Data access layer	All SQL (bound parameters only), row-to-object mapping, transactions, file storage.	Contain business decisions.
Storage	SQLite database file and the folder of submission files, both on the server.	Be reachable from any client.

5.2.2 Component Diagram

Figure 5.1 shows the components, the two nodes they run on, and the dependencies between them.

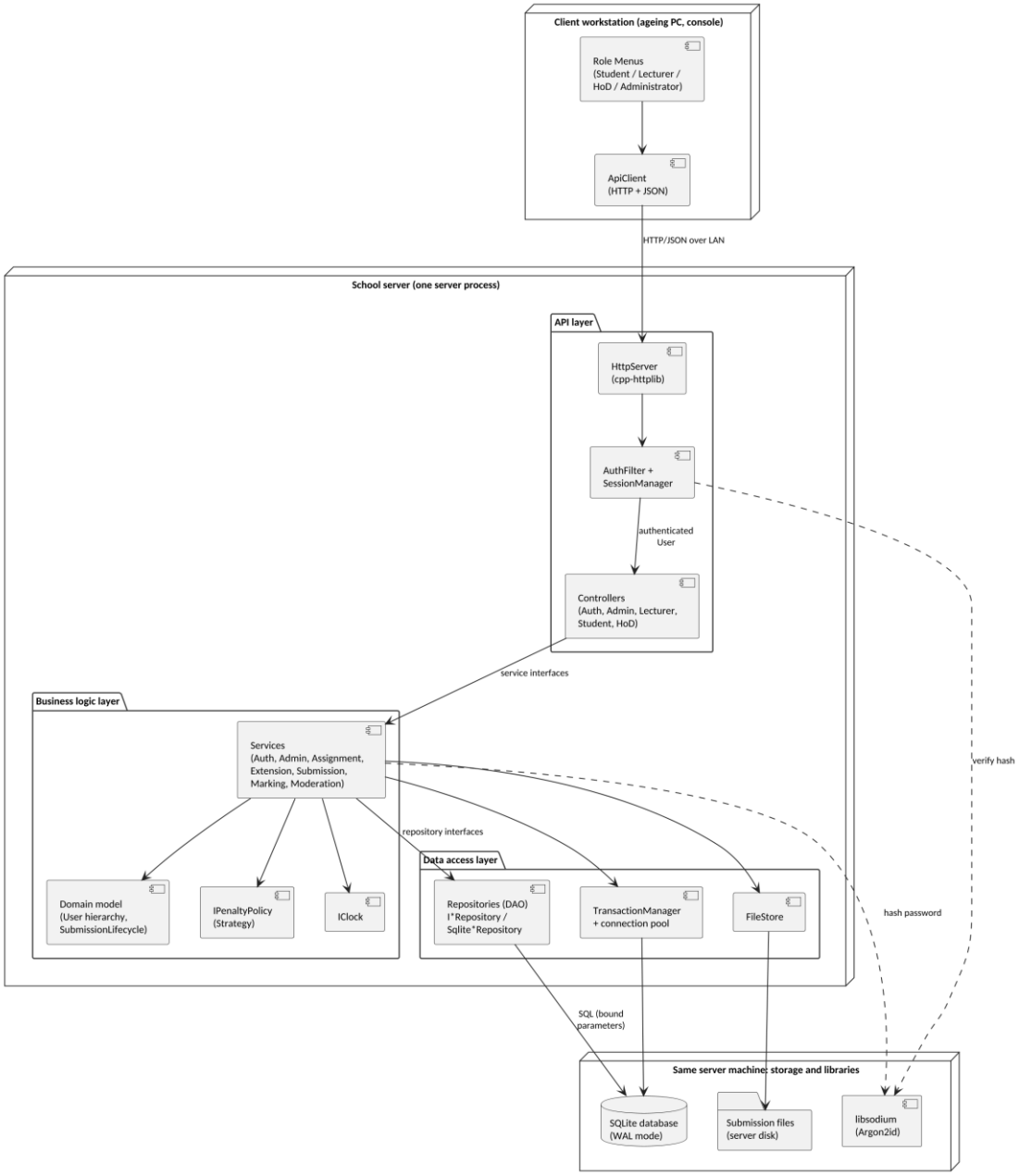


Figure 5.1: Component diagram of the layered client–server architecture.

Four points follow from the diagram:

- The client only sends requests and shows replies, so FR02 holds whatever a client displays.
- Only the repositories and the transaction manager touch SQLite, so bound parameters (NFR05) are enforced in one layer and can be inspected there.
- The penalty policy and the clock are injected into the services, so tests can replace them (NFR12) and the server time stays the only time source (AS1).
- libsodium is used only for hashing and verifying passwords and for generating session tokens.

5.2.3 Request Processing and Error Handling

Every request follows the same path, which keeps the controllers thin and the behaviour predictable:

1. The client sends an HTTP request with a session token. Submissions are sent as multipart form data so that a 5 MB file is not inflated by base64 encoding.
2. The AuthFilter looks up the session, rejects it if it has been idle for more than 15 minutes (FR03), and asks UserFactory to build the Student, Lecturer, HeadOfDepartment or Administrator object for that account.
3. The controller converts the request to a service call. The service checks that the role may perform the action and that the user is in scope (BR9), validates the input, and runs the work inside one transaction.
4. The service returns a result object, or throws one of the exceptions below. The controller turns either into a JSON reply.

Exceptions are the single error channel (NFR07). An unexpected exception is caught at the top of the request handler and returned as a generic 500 reply, so the server never stops because of one bad request, and no SQL text or file path is sent to the client.

Exception	Raised when	HTTP status
AuthenticationError	Invalid credentials, locked account, deactivated account, expired session. The message never says which credential was wrong (FR01).	401
AuthorisationError	The role may not do this, or the record is outside the user's scope (BR9).	403
ValidationError	Empty feedback, mark out of range, file missing or over 5 MB, empty reason, malformed input.	400
NotFoundError	The record does not exist, or exists but must not be revealed to this user.	404
ConflictError	Illegal status change (BR5), resubmission not allowed (BR4), or stale version (UC10 E5, UC17 E4).	409
DatabaseError	SQLite or file-system failure. The transaction is rolled back first.	500

5.2.4 Server Interface

The server exposes the endpoints below. Each maps to one service method, and each service method maps to one or more use cases. Backup and restore (NFR13) are not HTTP endpoints: they are options of the server program (`--backup <file>` and `--restore <file>`) that use SQLite's online backup API and can be run by the Administrator on the server machine.

Endpoint	Role	Use case	Service method
POST /api/login	Any	UC01	AuthService.login
POST /api/logout	Any	UC01	AuthService.logout
PUT /api/password	Any	UC01	AuthService.changePassword
POST, PUT /api/admin/users[/{id}]	Administrator	UC02	AdminService.createUser, updateUser, resetPassword
POST, PUT /api/admin/departments[/{id}]	Administrator	UC03	AdminService.saveDepartment
POST, PUT /api/admin/courses[/{id}]	Administrator	UC04	AdminService.saveCourse
PUT /api/admin/courses/{id}/lecturers	Administrator	UC05	AdminService.assignLecturers
PUT /api/admin/courses/{id}/enrolments	Administrator	UC06	AdminService.setEnrolments
POST /api/courses/{id}/assignments	Lecturer	UC07	AssignmentService.create

Endpoint	Role	Use case	Service method
PUT /api/assignments/{id}/extensions/{studentId}	Lecturer	UC08	ExtensionService.grantExtension
GET /api/assignments/{id}/submissions	Lecturer	UC09, UC11	SubmissionService.listForLecturer
PUT /api/submissions/{id}/mark	Lecturer	UC10	MarkingService.markSubmission
GET /api/student/courses	Student	UC12	AssignmentService.listForStudent
POST /api/assignments/{id}/submission	Student	UC13, UC14	SubmissionService.submit
GET /api/student/results	Student	UC15	SubmissionService.resultsForStudent
GET /api/hod/submissions?assignment={id}	Head of Department	UC16	ModerationService.listMarked
POST /api/submissions/{id}/approve	Head of Department	UC17	ModerationService.approve
PUT /api/submissions/{id}/adjust	Head of Department	UC18	ModerationService.adjust
POST /api/submissions/{id}/return	Head of Department	UC19	ModerationService.returnForRemark
POST /api/assignments/{id}/release	Head of Department	UC17	ModerationService.release
GET /api/hod/audit?submission={id}	Head of Department	UC16	ModerationService.auditFor

5.2.5 Concurrency and Data Integrity

The server handles requests on a pool of worker threads. SQLite runs in write-ahead-logging (WAL) mode, so readers do not block the writer. The data access layer keeps a small pool of read connections and **one** write connection protected by a mutex. Every data-changing operation starts with `BEGIN IMMEDIATE`, is kept short, and is committed or rolled back as a unit by the transaction manager (NFR08). A busy timeout of five seconds covers the rare case where a backup holds the lock. This is how risk R1 from the risk register is handled in the design.

Two users can act on the same submission at the same time, for example a Lecturer re-marking while the Head of Department approves. Each submission carries a `version` number that is increased on every update. A service passes the version it showed to the user, and the repository updates the row only if the version still matches (optimistic locking). A mismatch raises ConflictError, nothing is changed and the client reloads the record. This implements the "stale data" exception flows in UC10 and UC17.

Submission files are stored on the server disk and the database holds only the path, size and SHA-256 checksum. This keeps the database small (so backup stays within the one-minute limit of NFR13 and memory within NFR11) and gives the checksum needed to verify NFR03. The file is written first, then the database transaction records it. If the transaction rolls back, the service deletes the file, and a start-up sweep removes any file that has no database row.

5.2.6 Security Design

- **Passwords.** Hashed with Argon2id through libsodium (`crypto_pwhash_str`, interactive limits of 2 passes and 64 MiB). The encoded result contains its own random salt and

parameters, so the `password_hash` column is the only password data stored (NFR04). Verification uses `crypto_pwhash_str_verify`.

- **Hashing load.** Each hash needs 64 MiB of memory, so fifty simultaneous logins would need about 3 GB, far beyond the 300 MB server limit in NFR11. The design therefore allows at most two hash operations at a time, with other logins waiting in a queue. This caps hashing memory at 128 MiB. Whether the queue still meets the 2-second limit of NFR01 at 50 concurrent users cannot be known before measurement, so it is recorded as a new risk (R10, Section 5.7) and the load test must measure it.
- **Sessions.** A 256-bit random token from libsodium is held in an in-memory map with the last-activity time. Sessions end after 15 minutes of inactivity. Failed-login counts and lock times are stored in the database, so a lock survives a server restart (FR03).
- **Injection.** All SQL is written only in the Sqlite repositories and uses bound parameters (NFR05).
- **Authorisation.** Checked in the services on every call, in two steps: the role must permit the action (`User::isPermitted`), and the record must be in scope (assigned course, own department, own submission, BR9).

5.2.7 Design Rationale and Rejected Alternatives

Three alternatives were considered and rejected.

Alternative	Why it was considered	Why it was rejected
Two-tier desktop application: a client on every PC opens the SQLite file on a network share	Simplest to build. No server and no HTTP layer.	SQLite's file locking is unreliable on network file systems, which risks corruption with up to 50 users (NFR08, risk R1). Every PC would hold the whole database, including the password hashes, so FR02 and NFR04 could not be enforced. Business rules would be copied into each client.
Model–View–Controller on a single computer: one console program with SQLite on the same machine	The brief allows a single school computer, and MVC is a familiar style.	About 800 students and 50 staff cannot share one console. MVC organises a user interface, but here the "views" are on other computers, so a remote interface is needed anyway and the layered server gives that.
Web application or service-per-feature (microservices)	Scales well and gives browser access.	The brief puts web and mobile interfaces out of scope. Several processes would exceed the 2 GB client and 300 MB server limits (NFR11) and add deployment work for non-technical staff.

HTTP with JSON was chosen over raw sockets because cpp-httplib and nlohmann::json are header-only, need no installation steps, and let each endpoint be tested with an ordinary HTTP client. The costs of the chosen design are an extra layer, a little network latency (negligible on a LAN) and, in v1.0, traffic that is not encrypted. The last point is recorded as risk R11 in Section 5.7.

5.3 Class Design

5.3.1 Class Diagrams

The class design is split into four diagrams so that each can be read on one page: the users and organisation (Figure 5.2), the assessment domain (Figure 5.3), the business logic and Strategy pattern (Figure 5.4), and the repositories and infrastructure (Figure 5.5). Visibility is shown with +

(public) and - (private). `TimePoint` is `std::chrono::system_clock::time_point`, always in UTC (AS1). Optional values use `std::optional`.

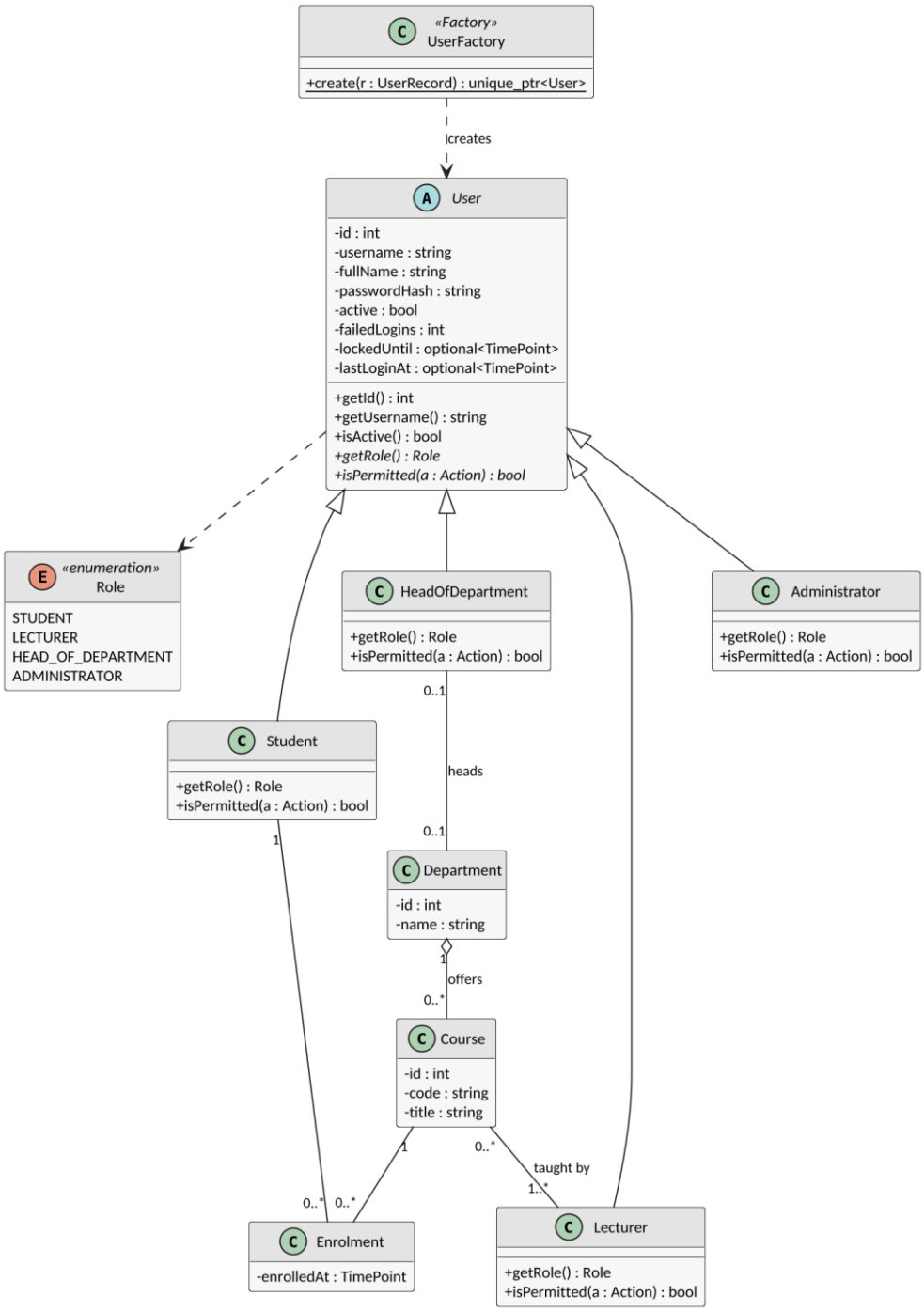


Figure 5.2: Class diagram – users, departments, courses and enrolment.

User is abstract. Its four subclasses override `getRole()` and `isPermitted(Action)`, so services ask the object what it may do and never test a role code themselves. The factory creates the right subclass from a stored account record. The table below shows the permission matrix that the four `isPermitted` implementations realise. A role permission is necessary but not sufficient: scope rules (BR9) are checked as well.

Action	Student	Lecturer	Head of Dept.	Admin.
Change own password	Yes	Yes	Yes	Yes
Manage users, departments, courses, lecturers, enrolments				Yes
Create assignment, grant extension, view submissions, mark		Yes (own courses)		
View courses and assignments, submit, resubmit, view released marks	Yes (own)			
Review department marks, approve, adjust, return, release, view audit			Yes (own department)	

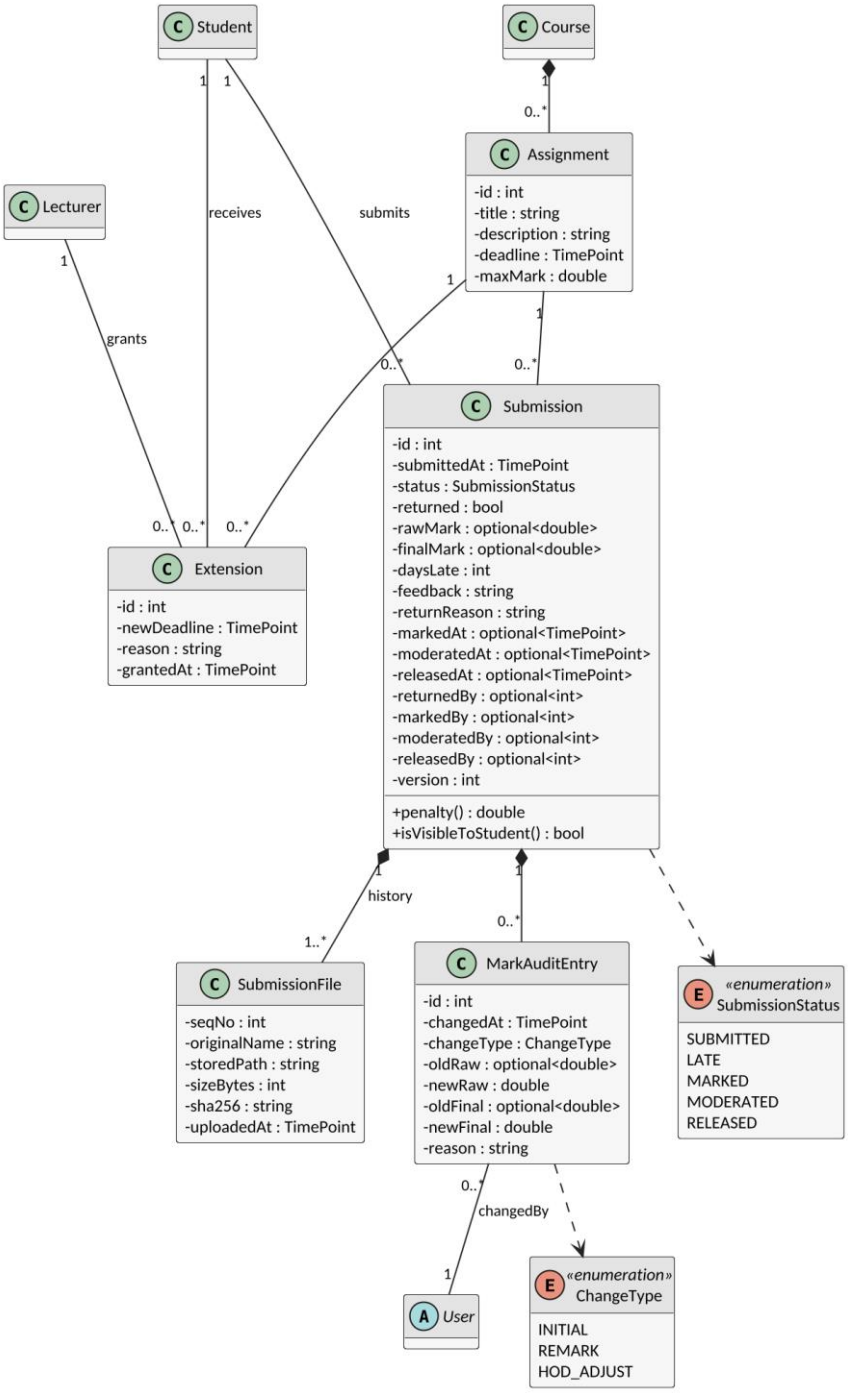


Figure 5.3: Class diagram – assignments, extensions, submissions and the mark audit.

Four modelling decisions in Figure 5.3 matter for the rest of the design:

- The effective deadline is **not stored**. It is computed from the assignment deadline and the student's Extension, if any (BR1), so the two can never disagree.
- A Submission is unique for each student and assignment. Replaced files stay as SubmissionFile objects with an increasing `seqNo`, which is the submission history required by BR4. The current file is the one with the highest `seqNo`.
- `penalty()` is derived as raw mark minus final mark. Only `daysLate` and `finalMark` are stored, as a snapshot taken when the mark is saved (Section 5.5.3).

- MarkAuditEntry has one entry for each mark change and holds the old and new raw and final marks together, which gives the "exactly one audit record" of NFR09.

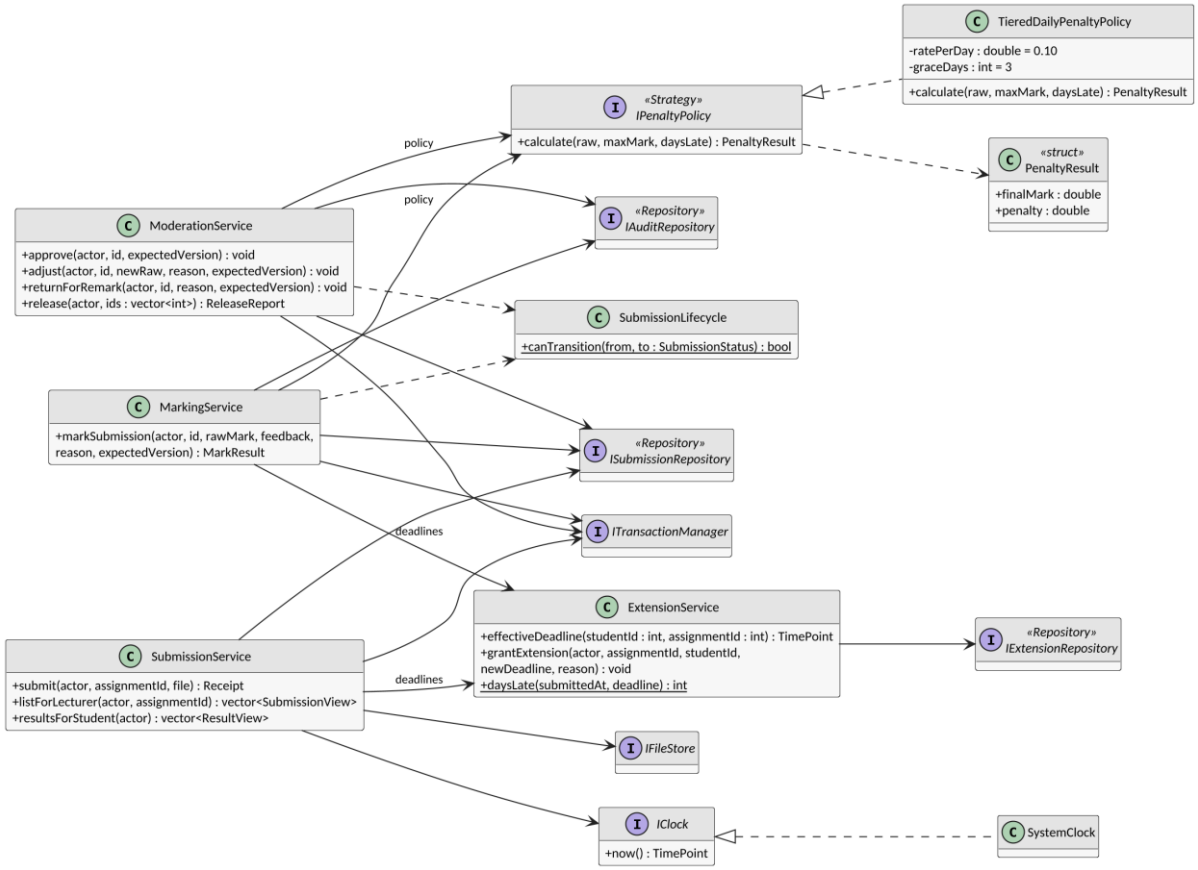


Figure 5.4: Class diagram – services, the late-penalty Strategy and the life-cycle guard.

Figure 5.4 shows the business logic. The two services that change marks, MarkingService and ModerationService, share one IPenaltyPolicy, so the penalty is calculated in exactly one place (FR18). The rule of BR3 is, in design terms:

```
daysLate(submittedAt, deadline):
    L = seconds(submittedAt - deadline)
    if L <= 0 then return 0 else return ceil(L / 86400)

TieredDailyPenaltyPolicy.calculate(raw, maxMark, d):
    if d == 0            then final = raw
    else if d <= 3       then final = max(0, raw - d * 0.10 * maxMark)
    else                      final = 0
    return (final, penalty = raw - final)
```

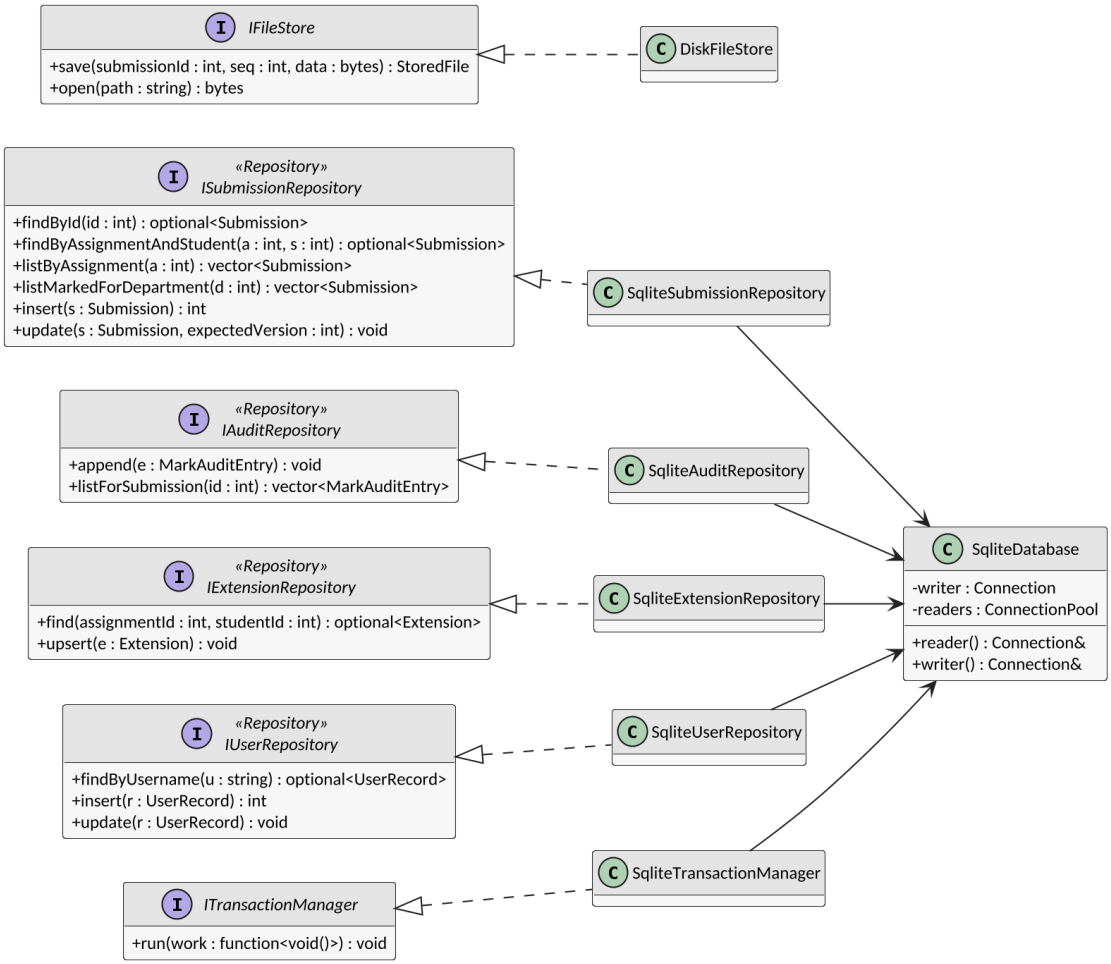


Figure 5.5: Class diagram – repository interfaces, SQLite implementations and infrastructure.

Figure 5.5 shows the data access layer. Each aggregate has its own narrow repository interface. `SqliteDatabase` owns the one write connection and the pool of read connections described in Section 5.2.5. The repositories for departments, courses, enrolments and assignments follow the same pattern as the four shown and are omitted from the figure for space.

5.3.2 Design Patterns Applied

Pattern	Where applied	Problem it solves	Why it was chosen
Strategy	IPenaltyPolicy with TieredDailyPenaltyPolicy, injected into MarkingService and ModerationService	The Head of Department's penalty rule (10% a day, three days, then zero) must be strict and applied the same way everywhere, yet its numbers may change.	The rate and the grace period are parameters of one class, and a different rule would be a new class with no change to the services. Because no service method or endpoint accepts a penalty, no user can override it (BR3, FR18).
Repository / DAO	I*Repository interfaces with Sqlite*Repository classes	Business code must not contain SQL, and SQL must always use bound parameters.	All SQL sits in one layer (NFR05). Services can be unit-tested with in-memory fakes (NFR12). The storage engine could change without touching business rules.

Pattern	Where applied	Problem it solves	Why it was chosen
Factory	UserFactory::create(UserRecord)	The server must build a Student, Lecturer, HeadOfDepartment or Administrator from a stored role string at login and on every request.	The choice of subclass is made in one place. Services receive a User and use polymorphism, so they contain no role switches.

Dependency injection supports all three, but it is a technique rather than one of the listed patterns. The Observer pattern was considered for FR28 (notice of newly released marks at login) and not used. FR28 can be answered by one query for submissions released since `last_login_at`, so an event mechanism would add complexity for no benefit.

5.3.3 SOLID Principles

Principle	How the design follows it
Single responsibility	MarkingService marks, ModerationService moderates, ExtensionService owns deadlines, SubmissionLifecycle owns legal status changes, and the penalty policy only calculates. A change to one rule touches one class.
Open / closed	A new penalty rule or a new storage back end is added as a new class that implements an existing interface, and no service changes. One limit is honest to state: a new user role also needs a new case in UserFactory.
Liskov substitution	Any User subclass can be used where a User is expected. `getRole()` is constant, and `isPermitted()` never throws and never grants more than the permission matrix allows. A common test runs against all four subclasses.
Interface segregation	Repositories are small and specific to one aggregate, instead of one large generic interface. IClock has one method and IFileStore has two, so a fake is trivial to write.
Dependency inversion	Services depend on abstractions (I*Repository, IClock, IPenaltyPolicy, ITransactionManager, IFileStore). The concrete classes are chosen only in the composition root.

5.4 Behavioural Models

5.4.1 State Machine Diagram: Submission Life Cycle

Figure 5.6 models the life cycle of a Submission. The status values are those of the SRS (Submitted, Late, Marked, Moderated, Released). The SRS says a returned submission "stays Marked and is flagged Returned", so Marked is drawn as a composite state with two sub-states, AwaitingModeration and Returned. The status stored in the database remains MARKED in both, and the flag `is_returned` distinguishes them.

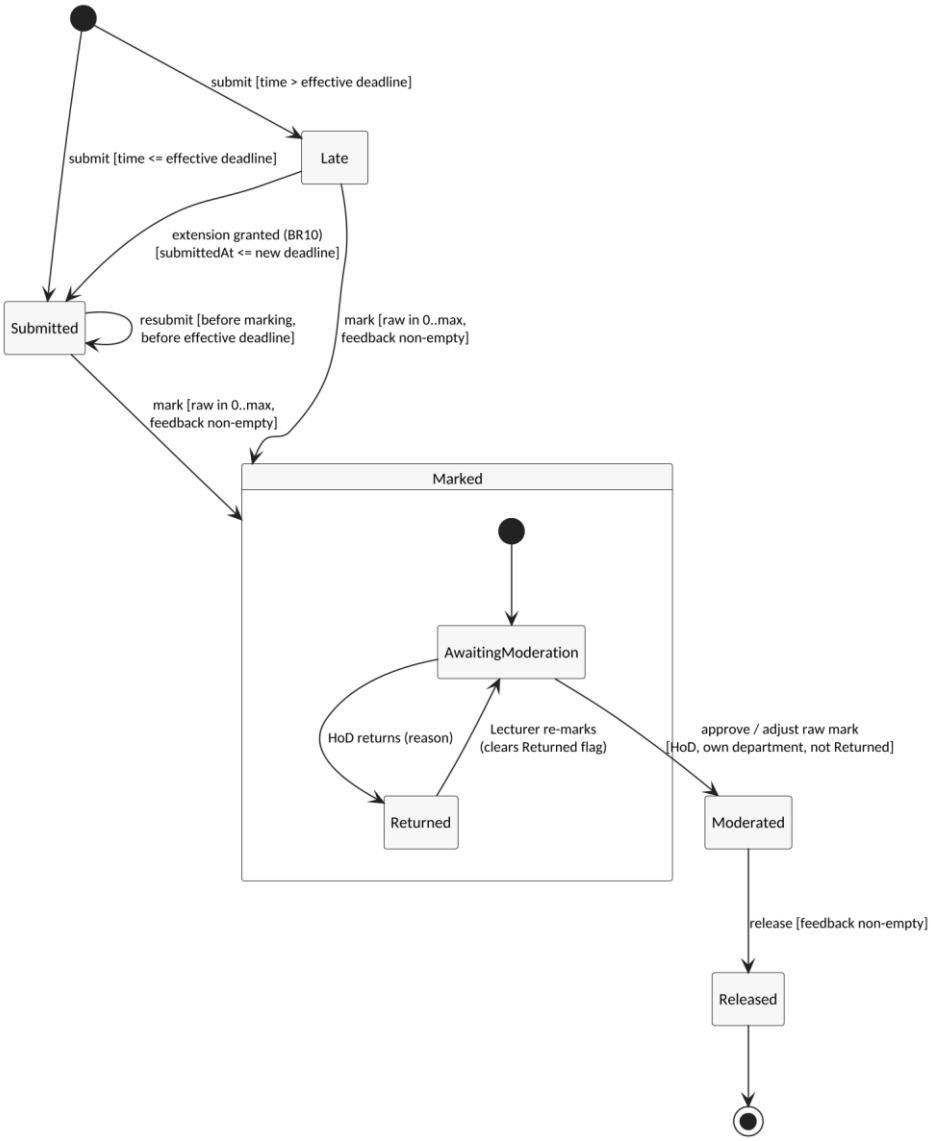


Figure 5.6: State machine diagram – Submission life cycle.

The transitions, with their guards and the rules they implement:

From	Event	Guard	To	Actor, rule
(start)	submit	time <= effective deadline	Submitted	Student, BR2
(start)	submit	time > effective deadline	Late	Student, BR2
Submitted	resubmit	not yet marked and time <= effective deadline	Submitted	Student, BR4
Late	extension granted	submission time <= new effective deadline	Submitted	Lecturer, BR10
Submitted or Late	mark	raw mark in 0..max and feedback not empty	Marked (AwaitingModeration)	Lecturer, BR5, BR7
Marked (AwaitingModeration)	return	reason not empty	Marked (Returned)	HoD, BR5, FR25
Marked (Returned)	re-mark	valid mark, feedback and reason	Marked (AwaitingModeration)	Lecturer, UC10 A4

From	Event	Guard	To	Actor, rule
Marked (AwaitingModeration)	approve, with or without adjusting the raw mark	own department; adjustment needs a reason	Moderated	HoD, FR21
Moderated	release	feedback not empty	Released	HoD, FR22, BR7

Every other event is rejected by `SubmissionLifecycle::canTransition` with a ConflictError (FR19). In particular a Moderated or Released submission cannot be marked or changed (BR5), and a Late submission cannot be replaced (BR4). The database adds a second line of defence through CHECK constraints (Section 5.5.5).

5.4.2 Activity Diagram with Swimlanes

Figure 5.7 follows one submission from the Student's submit action to the released mark. The four lanes are the Student, the System, the Lecturer and the Head of Department. The first loop covers an invalid file. The second loop covers the Head of Department returning a mark for re-marking: the flow goes back to the Lecturer and the cycle repeats until the mark is not returned. The diagram is consistent with the main flows of UC13, UC10 and UC17.

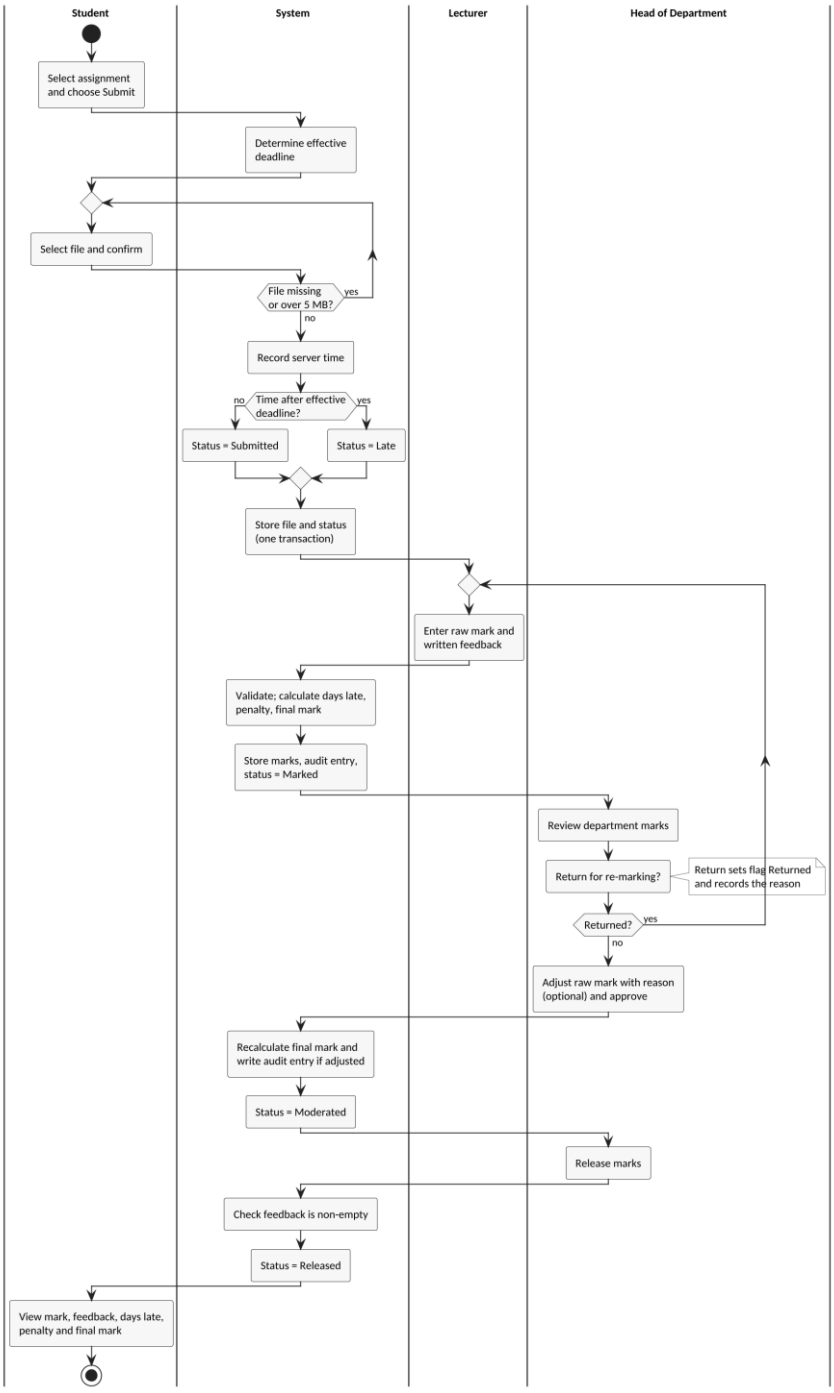


Figure 5.7: Activity diagram with swimlanes – from submission to released mark.

5.4.3 Sequence Diagrams

Figure 5.8 shows "Submit assignment" (UC13). The effective deadline and the server time decide the status. The first alternative shows the on-time and late paths: a late submission is still accepted (BR2). The second alternative separates a first submission, a permitted resubmission (status Submitted and still before the deadline, BR4) and a refused replacement. The file save and the database write happen inside one transaction.

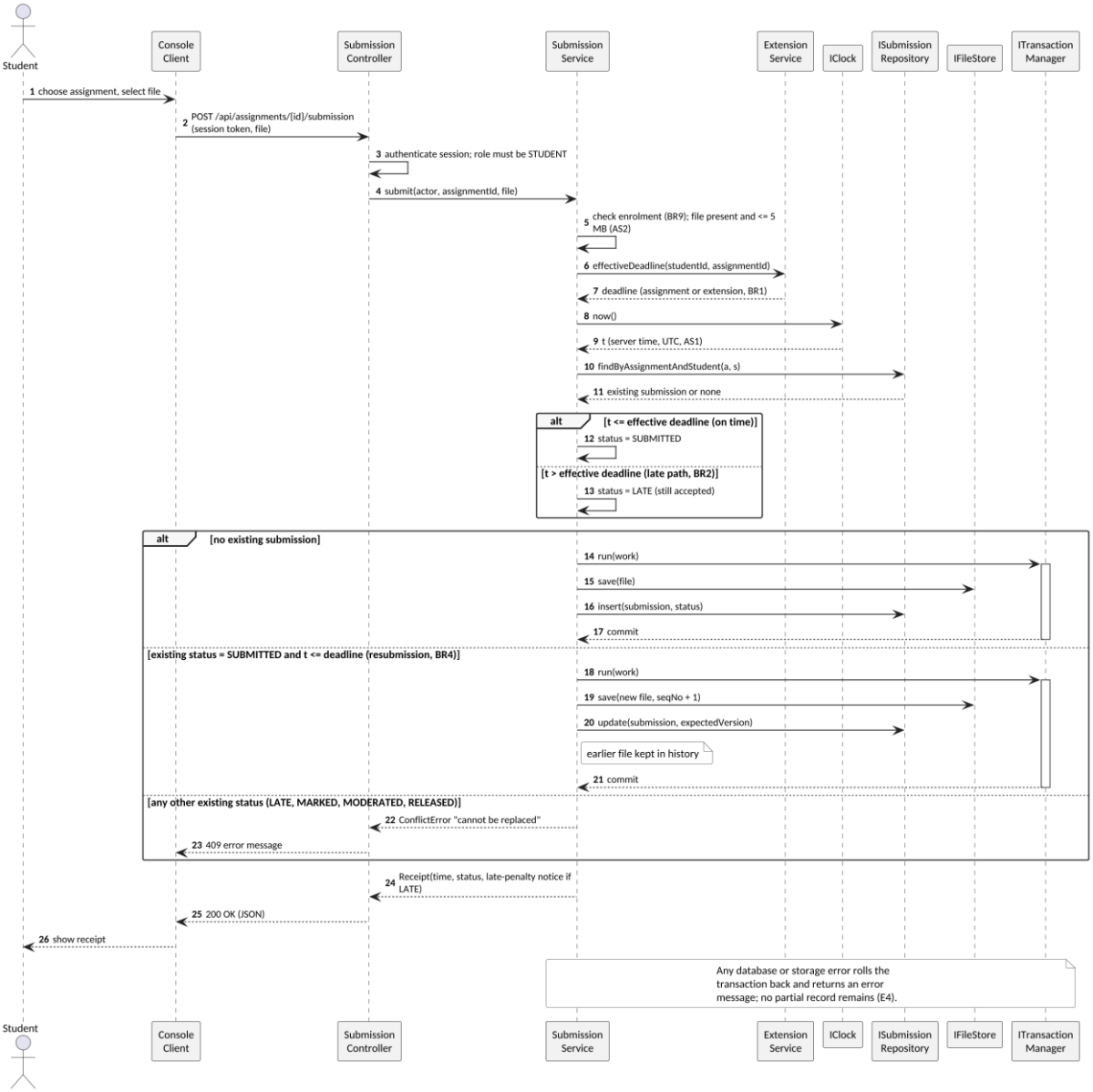


Figure 5.8: Sequence diagram – Submit assignment, including late and resubmission paths.

Figure 5.9 shows "Moderate and release marks" (UC17, with UC18 and UC19). Approval, adjustment and return each run in their own transaction for each submission. The adjustment calls the penalty policy again, because the final mark depends on the raw mark, and writes its audit entry in the same transaction as the change (NFR09). Release checks each submission for status Moderated and non-empty feedback, skips the ones that fail and reports them, as in UC17 E3.

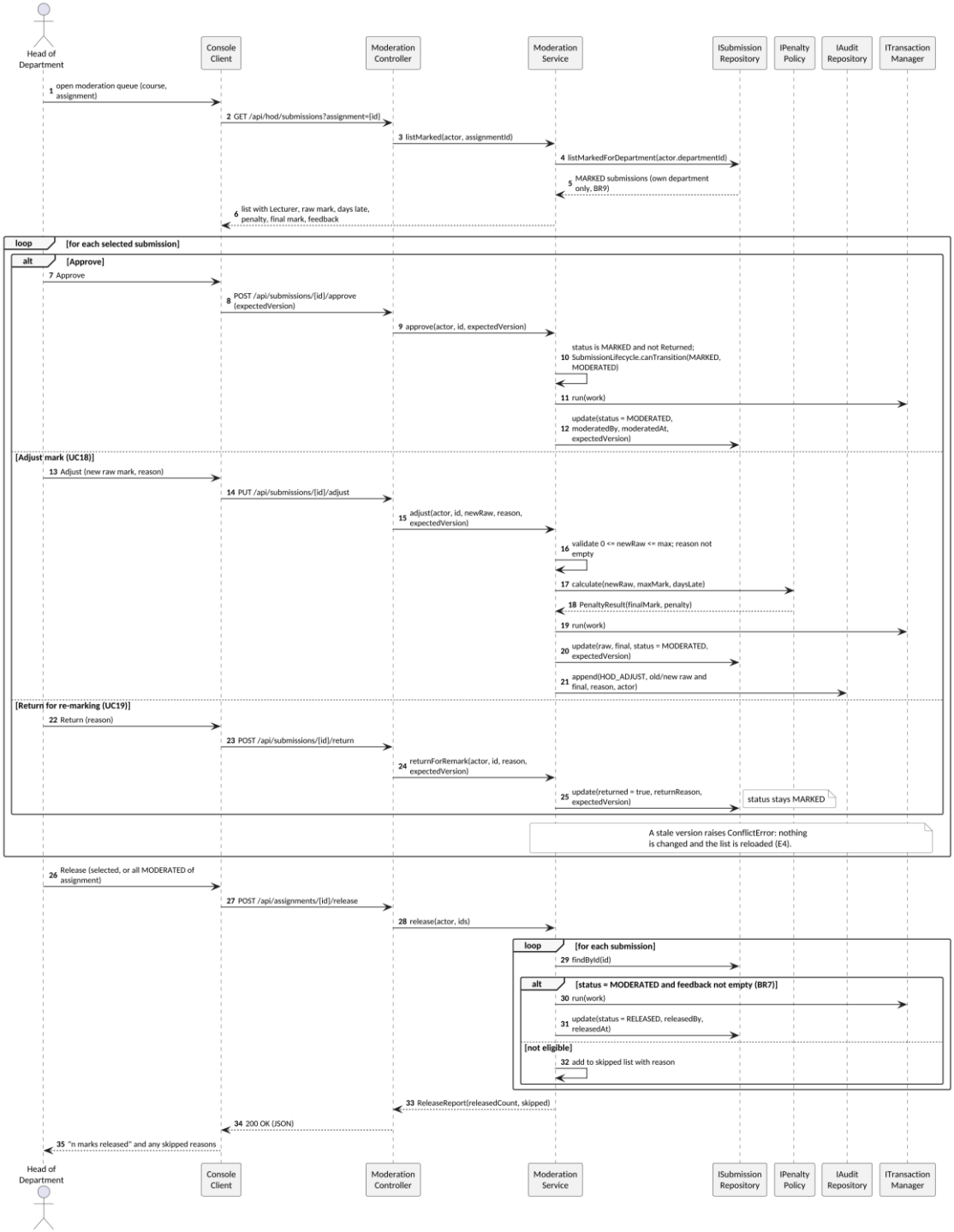


Figure 5.9: Sequence diagram – Moderate and release marks.

5.5 Database Design

5.5.1 Conventions

The database is SQLite 3 with `PRAGMA foreign_keys = ON` and WAL journal mode. Tables and columns use lower-case snake_case. Timestamps are ISO 8601 text in UTC, for example `2026-11-01T10:00:00Z`, because AS1 makes the server clock the only time source. Booleans are INTEGER 0 or 1 with a CHECK. Enumerations are TEXT with a CHECK listing the allowed values. Marks are REAL, because 10% of a maximum mark need not be a whole number.

5.5.2 Entity-Relationship Diagram

Figure 5.10 shows the ERD in crow's-foot notation. To keep it readable, the foreign keys from `app_user` that record who acted (`created_by`, `granted_by`, `returned_by`, `marked_by`, `moderated_by`, `released_by`) are listed in the data dictionary and not drawn.

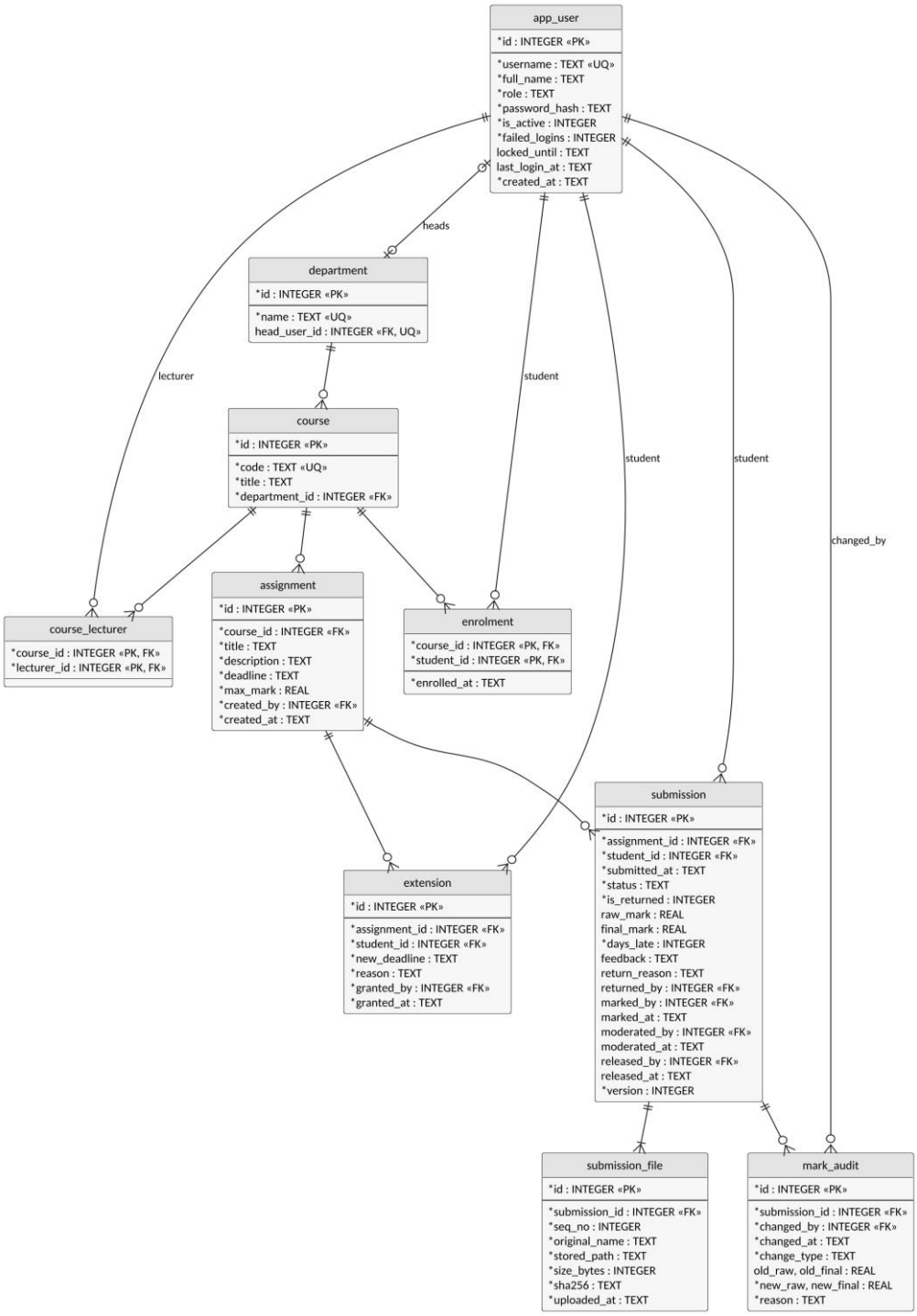


Figure 5.10: Entity-relationship diagram (Third Normal Form).

5.5.3 Normalisation

Each table was checked against Third Normal Form: every non-key column depends on the key, on the whole key, and on nothing but the key.

Table	Key	Main functional dependencies	Result
app_user	id (username is a candidate key)	id → every other column. Role is a plain attribute, not derived from anything else.	3NF
department	id (name, head_user_id unique)	id → name, head_user_id	3NF
course	id (code unique)	id → code, title, department_id. Department name is not repeated here.	3NF
course_lecturer	(course_id, lecturer_id)	No non-key columns. Resolves the many-to-many link.	3NF
enrolment	(course_id, student_id)	(course_id, student_id) → enrolled_at	3NF
assignment	id	id → course_id, title, description, deadline, max_mark, created_by, created_at	3NF
extension	id ((assignment_id, student_id) unique)	id → new_deadline, reason, granted_by, granted_at	3NF
submission	id ((assignment_id, student_id) unique)	id → all columns. Marks, status and the who/when columns describe this one submission. File details are in submission_file.	3NF, with one deliberate exception (below)
submission_file	id ((submission_id, seq_no) unique)	id → file name, path, size, checksum, time	3NF
mark_audit	id	id → all columns	3NF

Deliberate exception. `submission.final_mark` and `submission.days_late` could be computed from the raw mark, the maximum mark, the submission time and the effective deadline, so strictly they are derived data. They are stored as a snapshot taken when the mark is saved, for two reasons. First, the audit entry must record the old and new final mark exactly as they were shown to the user (BR8), even if the penalty parameters are later changed. Second, a released mark must never change because a later extension or policy change alters the inputs. The penalty itself is not stored: it is raw mark minus final mark.

5.5.4 Data Dictionary

app_user: all four roles in one table

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate key
username	TEXT	NOT NULL, UNIQUE, case-insensitive	Login name (FR01)
full_name	TEXT	NOT NULL	Display name
role	TEXT	NOT NULL, CHECK in the four roles	Discriminator for the User subclasses
password_hash	TEXT	NOT NULL	Argon2id encoded string with its own salt and parameters (NFR04)
is_active	INTEGER	NOT NULL, default 1, CHECK 0 or 1	0 = deactivated (FR04)
failed_logins	INTEGER	NOT NULL, default 0, >= 0	Consecutive failed logins (FR03)
locked_until	TEXT	NULL	End of the 5-minute lock (FR03)
last_login_at	TEXT	NULL	Used for the new-marks notice (FR28)

Column	Type	Constraints	Description
created_at	TEXT	NOT NULL	Creation time

department

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate key
name	TEXT	NOT NULL, UNIQUE	Department name
head_user_id	INTEGER	NULL, UNIQUE, FK to app_user	Head of Department. AdminService checks that the user has the role HEAD_OF_DEPARTMENT.

course

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate key
code	TEXT	NOT NULL, UNIQUE	Course code
title	TEXT	NOT NULL	Course title
department_id	INTEGER	NOT NULL, FK to department	Owning department (AS4)

course_lecturer and enrolment

Column	Type	Constraints	Description
course_lecturer.course_id	INTEGER	PK part, FK to course	Course taught
course_lecturer.lecturer_id	INTEGER	PK part, FK to app_user	Lecturer assigned (FR08)
enrolment.course_id	INTEGER	PK part, FK to course	Course enrolled in
enrolment.student_id	INTEGER	PK part, FK to app_user	Student enrolled (FR09)
enrolment.enrolled_at	TEXT	NOT NULL	Time of enrolment

assignment

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate key
course_id	INTEGER	NOT NULL, FK to course	Owning course
title	TEXT	NOT NULL	Title (FR10)
description	TEXT	NOT NULL, default empty	Description
deadline	TEXT	NOT NULL, ISO 8601 UTC pattern	Assignment deadline
max_mark	REAL	NOT NULL, > 0	Maximum mark (FR10, AS3)
created_by	INTEGER	NOT NULL, FK to app_user	Creating Lecturer
created_at	TEXT	NOT NULL	Creation time

extension

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate key
assignment_id	INTEGER	NOT NULL, FK to assignment	Assignment extended
student_id	INTEGER	NOT NULL, FK to app_user	Student who receives the extension
new_deadline	TEXT	NOT NULL, ISO 8601 UTC pattern	Extended deadline. ExtensionService checks it is later than the effective deadline (BR1).

Column	Type	Constraints	Description
reason	TEXT	NOT NULL, not blank	Reason (FR11)
granted_by	INTEGER	NOT NULL, FK to app_user	Lecturer who granted it
granted_at	TEXT	NOT NULL	Time granted
(table)		UNIQUE (assignment_id, student_id)	One extension per student and assignment. A newer one replaces the older (BR1).

submission

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate key
assignment_id, student_id	INTEGER	NOT NULL, FKs; UNIQUE together	One current submission per student and assignment
submitted_at	TEXT	NOT NULL	Server time of the latest submission (AS1)
status	TEXT	NOT NULL, CHECK in the five statuses	Life-cycle status (BR5)
is_returned	INTEGER	NOT NULL, default 0; 1 only if MARKED	Returned flag (FR25)
raw_mark	REAL	NULL, >= 0	Mark before penalty
final_mark	REAL	NULL, >= 0	Mark after penalty, a snapshot (Section 5.5.3)
days_late	INTEGER	NOT NULL, default 0, >= 0	Days late, a snapshot set when marked
feedback	TEXT	NULL; not blank once marked	Written feedback (BR7)
return_reason, returned_by	TEXT, INTEGER	NULL; returned_by FK to app_user	Why and by whom a mark was returned
marked_by, marked_at	INTEGER, TEXT	NULL until marked; FK to app_user	Who marked and when
moderated_by, moderated_at	INTEGER, TEXT	NOT NULL once Moderated	Who approved and when (UC17)
released_by, released_at	INTEGER, TEXT	NOT NULL once Released	Who released and when (UC17)
version	INTEGER	NOT NULL, default 1	Optimistic-locking counter (Section 5.2.5)

submission_file

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate key
submission_id	INTEGER	NOT NULL, FK to submission	Owning submission
seq_no	INTEGER	NOT NULL, >= 1; UNIQUE with submission_id	1 for the first file, then increasing. Highest = current.
original_name	TEXT	NOT NULL	File name as uploaded
stored_path	TEXT	NOT NULL, UNIQUE	Path on the server disk
size_bytes	INTEGER	NOT NULL, 1 to 5 242 880	Size, at most 5 MB (AS2)
sha256	TEXT	NOT NULL, 64 characters	Checksum (NFR03)
uploaded_at	TEXT	NOT NULL	Upload time

mark_audit: append-only

Column	Type	Constraints	Description
id	INTEGER	PK, autoincrement	Surrogate key
submission_id	INTEGER	NOT NULL, FK to submission	Submission whose mark changed
changed_by	INTEGER	NOT NULL, FK to app_user	Who (BR8)
changed_at	TEXT	NOT NULL	When (BR8)
change_type	TEXT	NOT NULL, INITIAL, REMARK or HOD_ADJUST	Kind of change
old_raw, old_final	REAL	NULL for the first mark	Old values (BR8)
new_raw, new_final	REAL	NOT NULL	New values (BR8)
reason	TEXT	NOT NULL, not blank	"Initial mark", or the stated reason (BR8)

5.5.5 SQL Schema

The schema below was executed on SQLite 3.45.1, and the constraints were checked with test inserts: a MARKED row with blank feedback, a returned flag on a non-MARKED row, a duplicate submission, an invalid status, a file over 5 MB, an update or delete on `mark_audit`, deletion of a user, a foreign-key violation and a zero maximum mark were each rejected.

```
PRAGMA foreign_keys = ON;
PRAGMA journal_mode = WAL;

CREATE TABLE app_user (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    username        TEXT    NOT NULL UNIQUE COLLATE NOCASE
                    CHECK (length(trim(username)) > 0),
    full_name       TEXT    NOT NULL CHECK (length(trim(full_name)) > 0),
    role            TEXT    NOT NULL
                    CHECK (role IN
('STUDENT','LECTURER','HEAD_OF_DEPARTMENT','ADMINISTRATOR')),
    password_hash   TEXT    NOT NULL,
    is_active       INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0,1)),
    failed_logins   INTEGER NOT NULL DEFAULT 0 CHECK (failed_logins >= 0),
    locked_until    TEXT,
    last_login_at   TEXT,
    created_at      TEXT    NOT NULL
);

CREATE TABLE department (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    name            TEXT    NOT NULL UNIQUE CHECK (length(trim(name)) > 0),
    head_user_id    INTEGER UNIQUE REFERENCES app_user(id)
);

CREATE TABLE course (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    code            TEXT    NOT NULL UNIQUE CHECK (length(trim(code)) > 0),
    title           TEXT    NOT NULL CHECK (length(trim(title)) > 0),
    department_id   INTEGER NOT NULL REFERENCES department(id)
);

CREATE TABLE course_lecturer (
    course_id       INTEGER NOT NULL REFERENCES course(id),
    lecturer_id     INTEGER NOT NULL REFERENCES app_user(id),
    PRIMARY KEY (course_id, lecturer_id)
) WITHOUT ROWID;

CREATE TABLE enrolment (
    course_id       INTEGER NOT NULL REFERENCES course(id),
    student_id      INTEGER NOT NULL REFERENCES app_user(id),
```

```
    enrolled_at    TEXT    NOT NULL,
    PRIMARY KEY (course_id, student_id)
) WITHOUT ROWID;

CREATE TABLE assignment (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id      INTEGER NOT NULL REFERENCES course(id),
    title          TEXT    NOT NULL CHECK (length(trim(title)) > 0),
    description    TEXT    NOT NULL DEFAULT '',
    deadline       TEXT    NOT NULL CHECK (deadline GLOB '????-??-??T??:??:??Z'),
    max_mark       REAL    NOT NULL CHECK (max_mark > 0),
    created_by     INTEGER NOT NULL REFERENCES app_user(id),
    created_at     TEXT    NOT NULL
);

CREATE TABLE extension (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    assignment_id  INTEGER NOT NULL REFERENCES assignment(id),
    student_id     INTEGER NOT NULL REFERENCES app_user(id),
    new_deadline   TEXT    NOT NULL CHECK (new_deadline GLOB '????-??-??T??:??:??Z'),
    reason         TEXT    NOT NULL CHECK (length(trim(reason)) > 0),
    granted_by     INTEGER NOT NULL REFERENCES app_user(id),
    granted_at     TEXT    NOT NULL,
    UNIQUE (assignment_id, student_id)
);

CREATE TABLE submission (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    assignment_id  INTEGER NOT NULL REFERENCES assignment(id),
    student_id     INTEGER NOT NULL REFERENCES app_user(id),
    submitted_at   TEXT    NOT NULL,
    status         TEXT    NOT NULL
                   CHECK (status IN ('SUBMITTED','LATE','MARKED','MODERATED','RELEASED')),
    is_returned    INTEGER NOT NULL DEFAULT 0 CHECK (is_returned IN (0,1)),
    raw_mark       REAL    CHECK (raw_mark >= 0),
    final_mark     REAL    CHECK (final_mark >= 0),
    days_late      INTEGER NOT NULL DEFAULT 0 CHECK (days_late >= 0),
    feedback       TEXT,
    return_reason  TEXT,
    returned_by    INTEGER REFERENCES app_user(id),
    marked_by      INTEGER REFERENCES app_user(id),
    marked_at      TEXT,
    moderated by   INTEGER REFERENCES app user(id),
    moderated_at   TEXT,
    released_by    INTEGER REFERENCES app_user(id),
    released_at    TEXT,
    version        INTEGER NOT NULL DEFAULT 1,
    UNIQUE (assignment_id, student_id),
    CHECK (is returned = 0 OR status = 'MARKED'),
    CHECK (status IN ('SUBMITTED','LATE')
           OR (raw_mark IS NOT NULL AND final_mark IS NOT NULL
               AND marked_by IS NOT NULL
               AND feedback IS NOT NULL AND length(trim(feedback)) > 0)),
    CHECK (status NOT IN ('MODERATED','RELEASED') OR moderated_by IS NOT NULL),
    CHECK (status <> 'RELEASED' OR released by IS NOT NULL)
);

CREATE TABLE submission_file (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id  INTEGER NOT NULL REFERENCES submission(id),
    seq_no         INTEGER NOT NULL CHECK (seq_no >= 1),
    original name  TEXT    NOT NULL,
    stored_path    TEXT    NOT NULL UNIQUE,
    size_bytes     INTEGER NOT NULL CHECK (size_bytes BETWEEN 1 AND 5242880),
    sha256         TEXT    NOT NULL CHECK (length(sha256) = 64),
    uploaded_at    TEXT    NOT NULL,
    UNIQUE (submission_id, seq_no)
);

CREATE TABLE mark_audit (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id  INTEGER NOT NULL REFERENCES submission(id),
    changed_by     INTEGER NOT NULL REFERENCES app_user(id),
    changed_at     TEXT    NOT NULL,
```

```
    change_type     TEXT    NOT NULL CHECK (change_type IN ('INITIAL','REMARK','HOD_ADJUST')),
    old_raw         REAL,
    new_raw         REAL    NOT NULL,
    old_final       REAL,
    new_final       REAL    NOT NULL,
    reason          TEXT    NOT NULL CHECK (length(trim(reason)) > 0)
);

CREATE INDEX ix_course_department    ON course(department_id);
CREATE INDEX ix_assignment_course    ON assignment(course_id);
CREATE INDEX ix_enrolment_student    ON enrolment(student_id);
CREATE INDEX ix_submission_student   ON submission(student_id);
CREATE INDEX ix_submission_status    ON submission(assignment_id, status);
CREATE INDEX ix_audit_submission     ON mark_audit(submission_id);

CREATE TRIGGER trg_audit_no_update BEFORE UPDATE ON mark_audit
BEGIN SELECT RAISE(ABORT, 'mark_audit is append-only'); END;

CREATE TRIGGER trg_audit_no_delete BEFORE DELETE ON mark_audit
BEGIN SELECT RAISE(ABORT, 'mark_audit is append-only'); END;

CREATE TRIGGER trg_user_no_delete BEFORE DELETE ON app_user
BEGIN SELECT RAISE(ABORT, 'accounts are deactivated, never deleted'); END;
```

The database enforces the rules that concern one row or one table: status vocabulary, "marked implies feedback", "returned implies MARKED", file size, uniqueness, and the append-only audit. Rules that compare values in different tables cannot be written as CHECK constraints, so they are enforced in the services and covered by tests: a mark must not exceed the assignment's maximum mark, an extension must be later than the effective deadline, the Head of Department must have that role, and a Lecturer must be assigned to the course (BR9).

5.5.6 Mapping Between Classes and Tables

Class (Section 5.3)	Table	Notes
User, Student, Lecturer, HeadOfDepartment, Administrator	app_user	Single-table inheritance with the `role` column as discriminator.
Department, and the "heads" association	department	The association becomes `head_user_id`, nullable and unique.
Course	course	`department_id` implements the "offers" association.
Course – Lecturer association (many-to-many)	course_lecturer	Join table.
Enrolment (association class)	enrolment	Composite primary key, with its own attribute `enrolled_at`.
Assignment	assignment	`course_id` implements the composition from Course.
Extension	extension	Unique on (assignment_id, student_id).
Submission	submission	Optional attributes become nullable columns. `…By` attributes become foreign keys to app_user.
SubmissionFile	submission_file	Implements the composition "history".
MarkAuditEntry	mark_audit	`changedBy` becomes `changed_by`.
Enumerations (Role, SubmissionStatus, ChangeType)	TEXT columns	Values restricted by CHECK constraints.
PenaltyResult, IPenaltyPolicy, Session, Receipt, view objects	(none)	Not persisted: results of calculations, or held in memory.

The differences between the class diagrams and the tables are deliberate:

- **One table for four subclasses.** The subclasses differ in behaviour, not in data, so separate tables would only add joins at every login. The discriminator plus the Factory pattern keeps the object model polymorphic.
- **Many-to-many links become tables.** Relational tables cannot hold a collection, so Course–Lecturer and Course–Student need join tables. Enrolment is a class in the model because it has its own attribute.
- **Derived values.** `penalty()` and the effective deadline are not stored. `final_mark` and `days_late` are stored as snapshots, for the reasons given in Section 5.5.3.
- **Types.** TimePoint becomes ISO 8601 text, `optional<T>` becomes a nullable column, and `bool` becomes INTEGER 0 or 1, because SQLite has no native types for these.
- **Not stored.** Sessions live only in server memory, so that a restart ends every session and no token is ever written to disk.

5.6 Traceability: Requirements to Design

This table extends the first links of the traceability matrix from Section 4. It connects each functional requirement to its use case and to the design elements that realise it. Source files and test cases will be added in Phases 4 and 5, and the complete matrix will be placed in the appendices.

FR	UC	Classes and components	Tables
FR01	UC01	AuthService, SessionManager, UserFactory, IUserRepository	app_user
FR02	All	AuthFilter, User::isPermitted, scope checks in every service	app_user
FR03	UC01	AuthService, SessionManager	app_user
FR04	UC02	AdminService, UserFactory, IUserRepository	app_user
FR05	UC02, UC01	AdminService, AuthService	app_user
FR06	UC03	AdminService, IDepartmentRepository	department
FR07	UC04	AdminService, ICourseRepository	course
FR08	UC05	AdminService, ICourseRepository	course_lecturer
FR09	UC06	AdminService, IEnrolmentRepository	enrolment
FR10	UC07	AssignmentService, Assignment, IAssignmentRepository	assignment
FR11	UC08	ExtensionService, Extension, IExtensionRepository	extension
FR12	UC09	SubmissionService.listForLecturer, ExtensionService	submission, extension
FR13	UC10	MarkingService, SubmissionLifecycle, IPenaltyPolicy, IAuditRepository (Figures 5.4, 5.7)	submission, mark_audit
FR14	UC12	AssignmentService.listForStudent, ExtensionService.effectiveDeadline	enrolment, assignment, extension
FR15	UC13	SubmissionService, IFileStore, IClock (Figure 5.8)	submission, submission_file
FR16	UC14	SubmissionService (Figure 5.8)	submission, submission_file
FR17	UC15	Submission::isVisibleToStudent, SubmissionService.resultsForStudent	submission

FR	UC	Classes and components	Tables
FR18	UC21	IPenaltyPolicy, TieredDailyPenaltyPolicy, ExtensionService::daysLate	submission (days_late, final_mark)
FR19	All	SubmissionLifecycle (Figure 5.6), CHECK constraints	submission
FR20	UC16	ModerationService.listMarked, ISubmissionRepository	submission, assignment, course
FR21	UC17, UC18	ModerationService.approve, adjust (Figure 5.9)	submission, mark_audit
FR22	UC17	ModerationService.release (Figure 5.9)	submission
FR23	UC22	IAuditRepository, ITransactionManager, append-only triggers	mark_audit
FR24	UC11	SubmissionService.listForLecturer, Submission status and returned flag	submission
FR25	UC19	ModerationService.returnForRemark	submission
FR26	UC16	ModerationService.auditFor, IAuditRepository	mark_audit
FR27	UC15	Submission::penalty, ResultView	submission
FR28	UC15	SubmissionService.resultsForStudent, User.lastLoginAt	submission, app_user

5.7 Design Review for Gate 2

5.7.1 Coverage of the Phase 3 Requirements

Phase 3 requirement in the brief	Where it is met
Architecture: style, justification, component diagram, rejected alternative	5.2.1 to 5.2.7, Figure 5.1, Table of rejected alternatives
Class diagram with visibility, types, multiplicities, inheritance, interfaces	5.3.1, Figures 5.2 to 5.5
At least two design patterns, why chosen, SOLID	5.3.2 (Strategy, Repository/DAO, Factory), 5.3.3
State machine diagram for the Submission life cycle	5.4.1, Figure 5.6
Activity diagram with swimlanes, submission to released mark	5.4.2, Figure 5.7
Sequence diagrams: Submit assignment (late and resubmission paths), Moderate and release marks	5.4.3, Figures 5.8 and 5.9
ERD in Third Normal Form, data dictionary, SQL schema, mapping to classes	5.5.2 to 5.5.6, Figure 5.10

5.7.2 Points Raised by the Design for the SRS

Designing in detail exposed three points that the SRS does not settle. They are listed so they can be decided at the Gate 2 review, and the SRS updated if they are accepted.

ID	Point	Proposed resolution
DA1	BR10 re-evaluates status and penalty when an extension is granted to a student whose submission is not yet marked. It does not say what happens if the submission is already Marked or later.	ExtensionService refuses an extension for a submission that is already Marked, Moderated or Released (ConflictError). Otherwise the effective deadline would change while the stored mark did not. Add this sentence to BR10.

ID	Point	Proposed resolution
DA2	The SRS requires protected passwords (NFR04) but says nothing about protecting them in transit. In v1.0 the client and server talk over plain HTTP on the school LAN.	Accept for v1.0 because the LAN is closed and offline (NFR06), record it as risk R11, and consider TLS for v1.1.
DA3	The SRS does not say whether marks may be fractional or how many decimal places are kept. 10% of a maximum of 25 is 2.5.	Marks and maximum marks are stored as REAL and shown with at most two decimal places. The penalty is calculated before rounding.

5.7.3 Risk Register Update

The brief requires the risk register to be updated at each phase gate. The changes below follow from the design work. Ratings use the scale of Section 3.3 (likelihood × impact, each 1 to 5). The Gate 2 review itself has not yet taken place, so the re-ratings of R3 and R8 are proposals for that review.

ID	Change	Likelihood	Impact	Exposure	Reason and mitigation
R3	Re-rate (proposed)	2	5	10	All Phase 3 artefacts are complete, so the time-pressure risk has mostly passed. It stays above the lowest level until the review approves the design.
R8	Re-rate (proposed)	1	3	3	The class diagram and ERD were designed together, and Section 5.5.6 documents every difference.
R10	New (technical)	3	4	12	Argon2id needs 64 MiB for each hash, which could break NFR01 (2 s login) or NFR11 (300 MB) at 50 concurrent users. Cap concurrent hashing at 2 (Section 5.2.6) and measure in the load test. If the limit fails, lower the Argon2id memory setting and document the trade-off. Owner: Lebohang Lebete.
R11	New (technical)	2	4	8	Passwords and marks travel unencrypted on the school LAN (DA2). Keep the server on an isolated LAN, state the limit in the installation guide, and consider TLS in v1.1. Owner: Lebohang Lebete.
R12	New (project)	3	3	9	The brief reduces marks where code does not follow the design. Review the design before coding starts, name classes and tables exactly as in this section, and extend the traceability matrix in Phases 4 and 5. Owner: Lebohang Lebete.